

Presentation Script — Review 1

RoadAssist · SWE4004 Cloud Computing and Applications · Dr. Nagendra Panini Challa 31 slides · 15 minutes speaking · 4 presenters · Q&A after

Speaking Order

#	Presenter	Slides	Section	Time
1	P Sai Nirisha Chowdary (24MIC7122)	1 – 6	Problem and Solution	3:00
2	T V S Jignesh (24MIC7190)	7 – 12	Cloud Foundation — Modules 1, 2, 3	3:15
3	V Saatwik Sairaam (24MIC7131)	13 – 19	Cloud Architecture — Modules 4, 5	3:45
4	G Parthavi (24MIC145)	20 – 26	Security — Module 6, and the Prototype	3:30
5	Nirisha (<i>returns to close</i>)	27 – 31	Output, Roadmap, Conclusion	1:30

Why this split: each person presents the part of the system they actually built. If a panel member asks a follow-up, the person at the podium is the person who can answer it. Nirisha opens and closes because she owns the user-facing product — the story starts and ends with the person on the roadside.

How to read this script

- Text in **normal type** is what you say. It is written to be spoken, not read silently — short sentences, one idea each.
- Text in *[square brackets and italics]* is a stage direction. Do not say it.
- Bold** inside speech marks a word to stress.
- You do not have to say it word for word. Learn the shape of it, then speak naturally. But do learn the **first and last sentence of your section by heart** — those are the two moments nerves show.

SEGMENT 1 — NIRISHA · Slides 1–6 · 3:00

Slide 1 — Title · 25 sec

[Stand still. Wait for the room to settle. Do not start until it is quiet.]

Good morning, sir. We are Team RoadAssist — I am Nirisha, and with me are Saatwik, Jignesh and Parthavi.

Our project is **RoadAssist**: a cloud-native, offline-first roadside assistance platform for India.

Our tagline is: *one platform, every vehicle, every phone, every road*. Over the next fifteen minutes we will show you why that sentence needs a cloud architecture to be true.

[Advance.]

Slide 2 — Agenda · 15 sec

We will move through six things: the problem, our idea, the cloud foundation, the cloud architecture, cloud security, and finally the working prototype.

Sections three, four and five map directly onto the six modules of this course.

[Advance.]

Slide 3 — The Problem · 40 sec

Imagine a vehicle breaking down at night, on a national highway, with no signal.

That is not a rare scenario. India has **sixty-three lakh** kilometres of road — the second largest network in the world — and most of it sits outside any organised service network. We record roughly **one point seven lakh** road deaths a year, the highest in the world. And a significant share of highway deaths are attributed not to the crash itself, but to how long help took to arrive.

Fewer than one in five vehicle owners has access to organised roadside assistance today.

[Point at the four failure boxes.]

And the reasons existing solutions fail are consistent. They assume a network. They assume a smartphone. They assume you drive a car. And nobody is collecting the data that would tell an authority where these breakdowns cluster.

[Advance.]

Slide 4 — Why Existing Solutions Do Not Work · 30 sec

We looked carefully at what already exists, and we want to be fair about it — each of these solves one slice well.

Insurance and manufacturer assistance is genuinely useful, but it is bundled with car policies and needs a call centre. Local mechanic contacts are cheap and familiar, but there is no verification, no pricing and no accountability. Ride-hailing apps have excellent dispatch technology, but they assume constant connectivity.

[Pause on the dark band.]

The gap is the intersection. **No platform today covers every vehicle class, works without connectivity, and reaches a feature phone.** That intersection is our project.

[Advance.]

Slide 5 — Our Idea · 40 sec

RoadAssist is a cloud-hosted platform that connects any stranded vehicle to the nearest verified mechanic or emergency responder — over the internet when it exists, and over SMS, a voice menu, or on-device intelligence when it does not.

Four pillars. **Every vehicle** — cars, bikes, autos, trucks, tractors and EVs, with diagnosis that knows the difference. **Every phone** — a full app, and complete access from a feature phone. **Every road** — offline-first, so it keeps working with zero signal. **Every second** — automatic crash detection that escalates in under ten seconds.

[Point at the bottom band.]

And this is the important part for this course. Demand is unpredictable and spread across a subcontinent. The emergency path must never go down. And four students cannot own physical infrastructure. **Cloud is not a hosting choice here — it is the enabling condition.**

[Advance.]

Slide 6 — The Platform at a Glance · 30 sec

Here is the whole platform in one picture. Seven client types. Twelve independently scalable cloud services. Three availability zones. And zero connectivity required for the core journey.

Every amber node on that map is a vehicle that can request help. That is the scale we are designing for — not one city.

[Handover. Turn slightly towards Jignesh.]

Jignesh will now take you through the cloud foundation — the concepts, the enabling technologies and the infrastructure mechanisms.

SEGMENT 2 — JIGNESH · Slides 7–12 · 3:15

Slide 7 — Divider: Cloud Foundation · 10 sec

[Take the clicker. Face the panel, not the screen.]

Thank you, Nirisha. This section covers Modules one, two and three — cloud concepts and models, the enabling technologies, and the infrastructure mechanisms.

[Advance.]

Slide 8 — Why This System Must Be Cloud-Based · 45 sec

Let us start with the question the panel should be asking: why cloud at all?

[Point at the chart.]

This is assistance demand across a day. At three in the morning we see about ninety requests. At six in the evening, nine hundred and fifty. That is a **ten-times swing within a single day** — and monsoon and festival periods push it further.

If we owned servers, we would have to buy for that six p.m. peak and let the hardware idle for the other twenty hours. On cloud we pay for the peak only while it lasts.

Add to that national reach through multiple regions, and high availability through replication across zones.

[Point at the risk card.]

We are not claiming cloud is free of risk. Vendor lock-in we answer with containers and open standards. Data sovereignty we answer by keeping every byte in Indian regions. Reduced control we answer with monitoring, service level objectives and audit logging.

[Advance.]

Slide 9 — Where We Sit in the Cloud Stack · 45 sec

Module one asks us to place ourselves precisely, so we will.

We are **both** a cloud consumer and a cloud provider.

We **consume** IaaS — virtual servers, virtual networks, block and object storage. We **consume** PaaS — managed Kubernetes, managed PostgreSQL, a managed message queue. And we **provide** SaaS — the RoadAssist app, the mechanic portal and the government dashboard are software delivered as a service to end users.

On deployment model, we are **hybrid, by necessity**. The main platform is public cloud, because it must be elastic. Government incident data and citizen medical records sit in isolated private infrastructure, because that is a legal requirement, not a preference. And crash detection runs at the edge, on the device itself, because that is the only way it survives having no signal.

[Point at the bottom band.]

On roles and boundaries: the provider owns the data centre and the hypervisor. We are the cloud consumer and the cloud service owner. Parthavi is our cloud resource administrator. Our organisational boundary ends at our virtual network — but our **trust** boundary extends into every managed service we call.

[Advance.]

Slide 10 — Cloud Enabling Technologies · 40 sec

Module two, five technologies, all five in our design.

Data centre technology — multi-region, multi-zone, in Indian regions. Virtualization — virtual servers, and containers on top of them. Web technology — REST over HTTPS, WebSocket for live tracking, and a progressive web app.

The two I want to draw out are these.

Multitenancy — one deployment serves citizens, mechanics, fleet operators and government officers, isolated by row-level security and separate encryption keys per tenant. And **service technology** — twelve loosely coupled services behind one API gateway, communicating over an event bus, so any one of them can be scaled or replaced without touching the others.

[Advance.]

Slide 11 — Cloud Infrastructure Mechanisms · 35 sec

Module three, five mechanisms.

Network perimeter — a virtual private cloud with private subnets and exactly one public ingress behind a web application firewall. **Virtual server** — Kubernetes nodes built from hardened images. **Cloud storage device** — block volumes for the database, object storage for photos and offline map packs. **Cloud usage monitor** — metrics, logs and traces for every request, which is what feeds our autoscaling. And **resource replication** across availability zones.

If you ask me which matters most for us, it is resource replication — because our emergency service has to survive losing an entire availability zone.

[Advance.]

Slide 12 — Virtualization, Multitenancy and the Perimeter · 25 sec

[Point to the first illustration, then the third.]

These three diagrams are drawn to the same isometric scale on purpose. The amber container inside the virtual server on the left is the **same unit** as the pod inside the perimeter on the right. It is one consistent model of the system, not three unrelated pictures.

[Handover.]

Saatwik will now take the cloud architecture.

SEGMENT 3 — SAATWIK · Slides 13–19 · 3:45

Slide 13 — Divider: Cloud Architecture · 10 sec

Thank you, Jignesh. Modules four and five — the fundamental cloud architectures, cloud operations, and the provider and consumer perspectives.

[Advance.]

Slide 14 — Fundamental Cloud Architectures Applied · 40 sec

Module four lists eight fundamental architectures. All eight are in our **target** architecture, and I want to be exact about which are built and which are designed.

Built and demonstrable today: workload distribution and resource pooling, in the form of the dispatch engine — a PostGIS nearest-neighbour search over a pool of providers, with the busy ones excluded. I can run that live.

Designed, not deployed: dynamic scalability, cloud bursting, elastic resource capacity, service load balancing, elastic disk provisioning and redundant storage. The design is on the slide with where each one sits. The cluster is not provisioned, so I will not show you an autoscaler and call it ours.

[Advance.]

Slide 15 — Dynamic Scalability: the design · 45 sec

[Say nothing for the first four seconds. Let the animation run one full cycle.]

That animation is our **scaling design**. I want to be clear before I explain it: this is not a running cluster. It is the architecture we would deploy, and I will tell you exactly what we have instead.

Notice the trigger we chose. We would scale on **queue depth per pod**, not CPU. Our workload is input-output bound — it waits on the database and on external APIs — so CPU would tell us almost nothing. That reasoning comes from our own measurements: our heaviest endpoint, dispatch, spends its time in PostGIS, not in the Node process.

The design is a Horizontal Pod Autoscaler on queue depth, a readiness gate so a new pod joins the load balancer only after it passes a health check, and a cluster autoscaler underneath for elastic resource capacity.

What is actually built: the readiness gate. Our `/health` endpoint returns 503 when the database is unreachable, so any orchestrator or load balancer takes the instance out of rotation. That is one real piece of this architecture, it is tested, and I can show you the 503.

[Point at the last card.]

And this is the part people forget in the design. When demand falls, the pods are removed and the nodes returned. **Releasing capacity is what makes the cost proportional.** Scaling up is easy. Scaling down is the discipline.

[Advance.]

Slide 16 — Load Balancing and Redundant Storage: the design · 40 sec

Two more architectures. Both are designed; neither is deployed. What backs them today is a single container with a documented backup procedure — I will say what that means at the end.

On the left, workload distribution and service load balancing. One inbound stream is spread across six healthy replicas. The balancer health-checks each replica continuously and removes any that fails, so a sick pod never receives traffic.

[Point right. Wait for Zone C to drop out.]

On the right, redundant storage. Three synchronised replicas across three availability zones — and watch, Zone C is lost. The remaining two keep serving, with **zero data loss**.

That is precisely why we are able to promise ninety-nine point nine nine percent availability on the emergency service.

[Advance.]

Slide 17 — System Architecture · 55 sec

This is the complete architecture, and I have deliberately banded it by cloud service layer.

[Trace top to bottom with your hand.]

At the top, the **SaaS** layer — what our users touch. Seven clients, including the feature phone on the right, which reaches us through the telecom gateway.

Everything funnels into **one** API gateway. That is the only thing on this diagram reachable from the internet. It does TLS termination, the web application firewall, layer-seven load balancing and rate limiting.

Below it, the **PaaS** layer — twelve services as auto-scaling, stateless pods on managed Kubernetes, replicated across three availability zones.

At the bottom, **IaaS** — PostgreSQL with multi-zone replicas, Redis as our resource pool, object storage, the event bus, and the usage monitor.

[Point at the red box.]

One thing I want you to notice. The emergency service is red because it is deployed **separately** — its own node pool, its own database connections, its own quota. If everything else on this diagram fails, an SOS

still goes through. Life-safety needs a different reliability class from bookings.

[Advance.]

Slide 18 — Migration and Scheduling · 40 sec

Module four also covers cloud operations.

On migration: we use live migration so workloads move between hosts with no downtime, rolling deployments so pods are replaced in batches, and an expand-migrate-contract pattern so a database schema change stays backward compatible with the code already running.

On scheduling, we do both kinds. **Static** — nightly model training and analytics run on a fixed schedule on cheaper interruptible capacity. **Dynamic** — assistance requests are scheduled in real time by queue depth, mechanic availability and geography.

[Point at the timeline.]

Here is cloud bursting as a **modelled** evening — a worked scenario, not telemetry from a running system. Five p.m., six pods, baseline. Rain begins, queue depth rises. Six twenty-five, the autoscaler would reach twenty-two. Six forty, we burst onto additional on-demand nodes. And at nine p.m. everything is released back to six. The shape of the curve is the point; the numbers are a projection, and I will not present them as measurements.

[Advance.]

Slide 19 — Provider and Consumer Perspectives · 35 sec

Module five asks us to look from both sides, and as I said earlier, we sit on both.

As a consumer, we select instance types, size node pools and configure the virtual network on IaaS. On PaaS we consume managed Kubernetes and managed PostgreSQL — no patching, no failover scripts. On SaaS we consume third-party services for SMS, payments and maps.

As a provider, we optimise our own SaaS as a single multi-tenant deployment with row-level isolation, we publish APIs so insurers and fleet operators can build on us, and we commit to a service level agreement — ninety-nine point nine percent for the platform, ninety-nine point nine nine for emergency.

The difference in mindset is this: as a consumer we optimise for cost and reliability. As a provider we optimise for tenant isolation and the promise we have made.

[Handover.]

Parthavi will now cover cloud security and take you through the prototype.

SEGMENT 4 — PARTHAVI · Slides 20–26 · 3:30

Slide 20 — Divider: Cloud Security · 10 sec

Thank you, Saatwik. Module six — cloud security.

[Advance.]

Slide 21 — Threats and Mechanisms · 50 sec

We approached this as threat first, control second.

[Point left.]

On the left, the threat agents and threats from Module six, each written as something that could actually happen to us. An anonymous attacker denying service against our public API. A trusted attacker — a **verified mechanic** — escalating privileges. A malicious insider reaching citizen medical records. Traffic eavesdropping on location data.

[Point right.]

On the right, the mechanisms. Encryption — TLS one point three in transit, AES two-fifty-six at rest, and column-level encryption for medical and identity data. Hashing — Argon2 for credentials, and a SHA-256 hash chain that makes our audit log tamper-evident. Digital signatures on the emergency handoff. A public key infrastructure issuing per-officer certificates for the government portal. Identity and access management with seven roles. Single sign-on. Default-deny security groups. And hardened virtual server images.

If you want one pairing: **insufficient authorization** — someone reading another user's booking by changing an ID in the URL — is answered by resource-scoped IAM, which checks not just the role but the ownership.

[Advance.]

Slide 22 — Defence in Depth · 40 sec

Two things worth drawing out.

[Point at the flow.]

Single sign-on. The user signs in once. A short-lived signed token is issued that carries the role but never the password. Every service verifies that signature **locally** — it does not call back to the auth service, which means authentication is not a single point of failure. Access is then decided per resource, so role plus ownership. And revocation is instant — signing out on one device kills the token family everywhere.

[Point at the bottom band.]

And hardened images. When a vulnerability is published we do not patch the running node. We **replace** it. That is only affordable because the nodes are virtual and the services on them are stateless — which is a cloud property, not a security product.

[Advance.]

Slide 23 — Divider: Prototype · 8 sec

Now to what we have actually built.

[Advance.]

Slide 24 — How It Works, End to End · 45 sec

Six steps. The vehicle breaks down, or the on-device model detects a crash. We capture symptoms by voice, photo or an OBD scan. The AI service returns a probable fault, a severity and the parts needed. A

geospatial query ranks nearby verified mechanics and sends offers. The user tracks the mechanic live. And the job closes with a digital invoice and a cashless payment.

[Point at the offline row.]

But this row is the one that matters. When there is no network: the diagnosis runs **on the phone** in under a hundred and twenty milliseconds. The request is queued locally and marked pending. A feature phone can complete the entire journey over SMS or a voice menu. And on reconnect the queue replays, with conflicts resolved server-side.

[Point at the emergency strip.]

And the emergency path, under ten seconds, end to end.

[Advance.]

Slide 25 — Offline-First · 40 sec

I want to spend a moment here, because this is what separates our project from a normal cloud application.

A roadside assistance platform that needs the internet fails at exactly the moment it is needed. So we treat the absence of a network as the **normal case**, not the exception.

Four layers. An on-device model under fifteen megabytes. A local write queue where SOS always jumps ahead. Full SMS, voice-menu and USSD access in eight languages for feature phones. And deferred sync where the server stays authoritative, so booking state can never corrupt.

To be clear about the relationship: the cloud does all the heavy work — training, dispatch, analytics, storage. But **nothing on the critical path depends on reaching it**.

[Advance.]

Slide 26 — LIVE: Crash to Responder · 45 sec

[Let the animation run. Do not talk over the first loop.]

This is the emergency path, and most of it is genuinely running — I will mark the one part that is not.

Detection happens on the handset, so it does not need a network — **built**, and I can demonstrate it with the network off. A thirty-second cancel window opens, because the model raises a **signal** and a human confirms an incident — **built, and enforced in code**: there is no transition from a model-detected signal to a dispatched responder that does not pass through a human confirmation. Emergency contacts are alerted with a live location link — **built**; the response tells you how many were *actually* reached, not how many we hoped to. The nearest responder is found by a PostGIS geospatial query — **built**, and I can run it.

The handoff to ERSS 112 is **not built**. It is designed as mutual TLS with a signed payload, and in this build it is a stub — our own API says so in the response: "*ERSS 112 handoff is stubbed in development — no real emergency service is contacted.*" Connecting it needs a government integration we do not have, and we would rather say that than let you believe an ambulance was called.

The same honesty applies to isolation: ADR-0005 designs the emergency service as a separate deployable with its own quota. Today it is a module in the same process. The rule it exists to protect — that a model can never dispatch — is enforced regardless, in the state machine.

[Point at the red banner. Slow down.]

And this is the sentence we want to leave you with from this section. **The AI can raise an incident. It can never dispatch one.** A false positive from a model must never send a real ambulance. That is enforced in code, not in policy.

[Handover.]

Nirisha will close with what we have built and where we go next.

SEGMENT 5 — NIRISHA · Slides 27–31 · 1:30

Slide 27 — Final Output · 40 sec

[Take the clicker.]

These are three screens from the working prototype — requesting help, the AI diagnosis with a confidence level and a clear "do not drive" verdict, and live tracking.

Let me be precise about what is complete, because the difference matters.

Running, and I can demonstrate every one of these: the database — 56 tables on PostgreSQL with PostGIS, migrated from empty and seeded, on Docker. Authentication end to end, with OTP login, refresh-token rotation and reuse detection, and role-based access. The full customer and mechanic journey, from breakdown to payment to review. Off-Grid Mode — SOS with no network, stored on the device, synchronised on reconnect with no duplicate. Real-time status over server-sent events. A CI pipeline that builds, typechecks, lints and runs every test suite on each commit. And a production Docker image, which we have built and run, with the whole test suite passing against it.

Not provisioned, and I will not claim it: the cloud infrastructure. There is no virtual network, no Kubernetes cluster and no multi-zone deployment. The architecture for all of it is designed and documented; none of it is deployed. What we have is a container that runs anywhere Docker runs, and an honest list of what we would need to take it further.

[Advance.]

Slide 28 — Technology Stack · 15 sec

Our stack, in four groups — cloud and infrastructure, backend and data, frontend and mobile, and AI and operations.

Every component is open source or a managed service on a free tier. Total infrastructure cost for this project so far: **zero rupees.**

[Advance.]

Slide 29 — Roadmap · 15 sec

Review one is complete at twenty-five percent. Review two takes us to seventy — all services deployed and auto-scaling, AI diagnosis live, the offline engine and SMS working end to end. Review three completes it with the emergency engine, the government dashboard, load and security testing, and production deployment with a disaster recovery drill.

[Advance.]

Slide 30 — Conclusion · 20 sec

To close.

We had a problem with unpredictable, national-scale demand — the architectural answer is dynamic scalability and cloud bursting, designed and not yet deployed. A service that must never fail — and here the answer we actually built is not in the cloud at all: it is on the device. Off-Grid Mode means an SOS survives the failure that really happens on a highway, which is the user's own connection. Sensitive personal and medical data — answered by encryption at rest, role-based access, break-glass with a tamper-evident audit chain, and a security suite of 72 attacks that all fail. And four students with no hardware budget — answered by a design that consumes IaaS and PaaS pay-as-you-go, and by building the whole platform so it runs with zero third-party accounts until we choose otherwise.

A cloud problem, solved with cloud architecture.

Thank you, sir. We are happy to take questions.

[Stop. Stand still. Do not fill the silence.]

Q&A — Prepared Answers

Whoever owns the area answers. If you do not know, say "we have not evaluated that yet, sir" — never invent a number.

#	Likely question	Who	Answer
1	Why not just use a monolith on one server?	Saatwik	A single server cannot absorb a ten-times daily demand swing without being sized for the peak and idling the rest of the day, and it cannot survive a zone failure. Both are hard requirements for us.
2	Which cloud provider, and why?	Parthavi	We are provider-agnostic by design — everything is containerised and defined in Terraform. We develop against a MeitY-empanelled Indian region because data localisation under the DPDP Act is a legal requirement.
3	How is this different from Uber or Ola?	Nirisha	Their dispatch technology is excellent, but it assumes constant connectivity and a smartphone. Our core journey completes with neither. That single constraint changes the whole architecture.
4	Is your AI actually trained, or is it rules?	Jignesh	For Review 1 the diagnosis endpoints are rule-based and we state that openly. Models replace the rules behind an unchanged contract in Review 2, and the rules remain the permanent fallback.
5	What happens if the cloud region goes down?	Saatwik	Services are spread across three availability zones, so a zone loss is absorbed automatically. A full region loss is our disaster recovery scenario — restore into a second region, which we drill in Review 3.
6	How do you keep costs down at scale?	Parthavi	Autoscaling that releases capacity, interruptible instances for batch work, self-hosted map tiles instead of per-tile licensing, and cost alerts tied to a per-user budget.
7	How do you stop a mechanic seeing another user's data?	Parthavi	Resource-scoped authorization — the check is role plus ownership, at the resource level, not just the route. We test it with negative access-control cases in our test suite.
8	Multitenancy — how is one tenant isolated from another?	Jignesh	Row-level security in the database, a separate encryption key per tenant, and jurisdiction scoping for government users. Aggregated government views additionally enforce k-anonymity.
9	Ten seconds for emergency — how did you measure that?	Parthavi	It is our target and our published service level objective for Review 3, measured at the 95th percentile from detection to responder notification. We will demonstrate the measurement, not just assert it.
10	How does a feature phone user pay?	Nirisha	Cash on completion, or a UPI collect request sent to the registered number. The SMS path deliberately never handles card details.
11	Why PostgreSQL and not a NoSQL database?	Saatwik	Our domain is relational and transactional — bookings, invoices and state transitions need real constraints. PostGIS also gives us geospatial queries natively, which dispatch depends on.
12	What is your biggest technical risk?	Saatwik	Offline synchronisation conflicts. We wrote the conflict resolution matrix before writing any sync code, and booking state is always server-authoritative so it cannot corrupt.
13	Is this scalable to all of India?	Saatwik	The architecture is. The honest constraint is mechanic supply density, which is a partnership problem rather than an engineering one.
14	What did you personally build?	Each	Answer for your own area only, in one sentence, and name the specific artefact.

Timing and Delivery

Checkpoint	Should be at	If you are behind
3:00	End of slide 6	Nirisha: cut the competitor detail on slide 4
6:15	End of slide 12	Jignesh: on slide 10, name the five and expand only multitenancy
10:00	End of slide 19	Saatwik: cut slide 18's migration list, keep the bursting timeline
13:30	End of slide 26	Parthavi: on slide 21 read only the left column and one pairing
15:00	End of slide 30	Nirisha: on slide 27 name the five items without expanding them

Delivery rules for all four

1. **Face the panel, not the screen.** Glance at the slide, then turn back. You already know what is on it.
2. **Pause on the animated slides.** Slides 15, 16 and 26 loop on their own. Let each run one full cycle in silence before you speak. Silence reads as confidence.
3. **Hand over by name.** "Jignesh will now take the cloud foundation." Then step back physically so the panel's attention moves with the clicker.
4. **Numbers slowly.** "Ninety-nine point nine nine" is four words. Do not rush them.
5. **Never say "basically", "actually", or "as you can see".** If they can see it, you do not need to say it.
6. **If you blank,** look at the slide and describe what is on it out loud. The words come back.
7. **If the panel interrupts,** answer, then say "coming back to the slide" and continue. Do not restart your section.

Before you walk in

- [] Deck open in **Presenter View** — your speaker notes are already in the file
- [] Slideshow tested once end to end, so the three animated slides are confirmed playing
- [] Backup demo video on the local disk, **not** in cloud storage
- [] Deck also copied to a pen drive and emailed to yourself
- [] Phone charged, with the prototype installed and airplane mode ready to demonstrate
- [] One printed copy of this script at the podium, one line per slide
- [] Laptop charger in the bag

A note on honesty. Slide 27 claims five things are complete. Present only what the team has genuinely finished. If one of the five is not done, change the slide before the review and say so plainly — a panel forgives incomplete work far more readily than a claim that does not survive a follow-up question.